Yuan (Merlin) Chang

Email: merlin.y.chang@gmail.com
Github: github.com/itis2010me
Mobile:+1 (530) 760 6690
Portfolio: itis2010me.github.io/itis2010me

Experience

Amazon Web Services (AWS) - Automated Reasoning Group — Seattle, WA

- *Software Development Engineer II* — *July 2026 - Present*
  Software Development Engineer I — *Feb 2025 - July 2026*
  - Built an AI agent tool call evaluation system using formal policies such as Dogwood to improve agent safety.
  - Raised the compute fleet scaling ceiling by 20x (3k to 60k) and cut per-unit scale-up time by 60x through a backend migration of a highly concurrent, distributed service to Firecracker microVMs.
  - Lowered the barrier for internal builders and scientists to adopt formal reasoning engines (SAT/SMT, Lean4) by designing, implementing and shipping remote stateful MCP servers.
  - Eliminated service outages caused by partial availability-zone dependency failures, using automatic Elastic Load Balancing driven by deep health checks.
  - Cut recurring on-call pages by hardening the service's handling of asynchronous AWS Elastic Container Service events.
  - Mentored junior engineers and interns through design and code review, scoping and guiding an intern project that converted to a full-time offer.

- **Amazon Web Services (AWS) - High Performance Computing** — Seattle, WA
  Software Development Engineer I — *Apr 2024 - Feb 2025*
  - Shipped AWS Parallel Computing Service (PCS), a public managed HPC service, as part of the launch team, building across the AWS Console frontend, the backend service APIs, and the workflows that provisioned customer compute clusters.

  Software Development Engineer Intern — *June 2023 - Sep 2023*
  - Designed internal RESTful APIs and orchestration that collapsed multi-step cluster configuration and job scheduling into single API invocations.
  - Shipped a public AWS CLI command exposing the managed HPC service directly to enterprise customers.

- **UC Davis Applied Mathematics Research Group** — Davis, CA
  Research Assistant & Software Engineer — *June 2021 - June 2022*
  - Developed automated reasoning pipeline that computed undiscovered Ramsey constants.
  - Reduced the search space by 6x through symmetry breaking and optimization techniques in preprocessing.
  - Achieved up to 10x improvement in SAT clause generation through computation optimizations.

Technical Skills

- **Languages** Java, TypeScript, Bash, Rust, Python
- **Distributed Systems** REST APIs, NoSQL, Firecracker microVMs, containerization, image-based deployment
- **Cloud & Build** AWS (Lambda, ECS, SQS, DynamoDB, ElastiCache, S3, ELB), build systems, CI/CD tooling
- **Agentic AI** MCP servers, agent skills, steering docs, Claude Code, Kiro

Education

- **University of California, San Diego** — La Jolla, CA
  Master of Science - Computer Science and Engineering — *2022 - 2024*
- **University of California, Davis** — Davis, CA
  Bachelor of Science - Computer Science, Minor in Mathematics; Graduated with Honors — *2018 - 2022*

Publications

- **Rado Numbers and SAT Computations (with J. A. De Loera and W. J. Wesley).**
  Proceedings of the 47th International Symposium on Symbolic and Algebraic Computation (ISSAC 2022). Pages 333-342, available online at https://dl.acm.org/doi/10.1145/3476446.3535494.

Projects

- **Computations with Rado Numbers and Degree of Regularity (Automated Reasoning, Theory of Computation, Number Theory, Combinatorics)**: Advancement in terms of Rado Numbers and the degree of regularity. Discovery of nearly 500 new mathematical constants. Results collected and formulated into research paper accepted into 2022 ISSAC conference. Co-author with Prof. Jesús De Loera and William Wesley. Github Link
- **Minimization of CA Wildfire Impact (Web Mining, Convex Optimization, KKT, Lagrangian, Data Analysis)**: Built a connectivity matrix of CA counties from humidity and fire probability, formulated wildfire prioritization as a constrained optimization problem, and solved it with integer programming over 2021 data to rank counties by financial, social and geological impact.
- **Terminal UI Board-game In Haskell (Haskell, Functional Programming, QuickCheck, Genetic Algorithm)**: TUI implementation of the classic board-game Mastermind in Haskell, using monadic and functional programming, with 10 QuickCheck property tests and a genetic-algorithm AI player built on minimax. Github Link